

Does Dynamic-Point Filtering Help When Texture Is Scarce?

A Controlled Study of ORB-SLAM2 Front-Ends in Synthetic Indoor Scenes

Zekui Xue

Department of Computer Science

University of Bath

Bath, United Kingdom

zekui.xue@bath.edu

Abstract—Dynamic-point filters are routinely added to feature-based visual SLAM, and several recent systems argue that removing dynamic features can leave too few static features in low-texture regions. So far, these systems have been evaluated only on texture-rich benchmark sequences. We present a controlled study that isolates this interaction. We render synthetic indoor sequences in which surface texture (four levels, quantified by FAST-corner density and image-gradient entropy) and scene dynamics (three levels) are varied factorially along identical camera trajectories, with stereo, RGB-D, ground-truth poses and dynamic masks. On this grid we compare ORB-SLAM2 without filtering, with an optical-flow and epipolar-residual filter (FLOW), and with a multi-view depth-consistency filter (GEOM), and report trajectory error, tracking completeness and surviving static features over five runs. Because the masks give per-keypoint ground truth, we also measure each filter’s dynamic-point precision and recall and its static-feature false-removal rate, so that mechanistic explanations can be tested directly. We do not propose a new filter. On 720 runs over 24 sequences, filtering helped mainly in the most dynamic cells; the benefit did not decline monotonically with texture, but at the lowest level filtering reduced tracking completeness, and ORB-SLAM2 never initialised in static L3 scenes. Contrary to our hypothesis, GEOM discarded more static keypoints than FLOW (median FRR 6.8% vs. 1.5% for RGB-D, 19.6% vs. 1.5% for stereo); its RGB-D advantage tracked dynamic-point recall and vanished in stereo mode. Data and code are available at <https://github.com/felixxue/texture-dynamics-slam>.

Index Terms—Visual SLAM, dynamic environments, low-texture scenes, ORB-SLAM2, synthetic data, evaluation

I. INTRODUCTION

Feature-based visual SLAM systems such as ORB-SLAM and ORB-SLAM2 [1], [2] estimate the camera pose from keypoints that are assumed to lie on static structure. Moving people and objects violate this assumption, and a large family of dynamic-SLAM front-ends therefore removes keypoints that are judged to be dynamic before, or during, tracking. Two classical cues for this decision are *optical flow with an epipolar consistency test*, as in the moving-consistency check of DS-SLAM [3], and *multi-view geometry*, as in the depth and parallax test of DynaSLAM [4]; both were built on ORB-SLAM2.

Filtering has a cost that is easy to overlook in texture-rich scenes: every removed keypoint, whether truly dynamic or a false positive, is one fewer constraint for pose estimation. When the scene offers few keypoints in the first place, for example white walls, uniform floors and plain furniture, the same filter may push the tracker below the number of static features it needs. Evaluation studies have shown that sparse feature-based systems such as ORB-SLAM2 and ORB-SLAM3 are severely affected in textureless environments [5]. Three recent systems state the interaction explicitly: IL-SLAM notes that dynamic removal creates the new problem of insufficient remaining point features and adds line features when needed [6]; SR-SLAM scores frame reliability partly by feature richness [7]; and SFSE-SLAM observes that “removing dynamic feature points often leaves insufficient static features in low-texture regions” and compensates with a density-driven adaptive threshold [8]. However, these systems are evaluated on TUM RGB-D [9] and Bonn [10] sequences (and, for SR-SLAM, on real robot runs), none of which varies texture in a controlled way. Whether, and by how much, the benefit of dynamic-point filtering shrinks as texture becomes scarce has, to our knowledge, not been measured.

This paper measures it. We hold scene geometry, camera trajectory and the motion of dynamic agents fixed, vary surface texture and the number of moving agents factorially in simulation, and run ORB-SLAM2 with no filter, a flow-based filter and a geometry-based filter in both stereo and RGB-D modes. **We do not propose a new filter**; our aim is to quantify, under controlled texture, when existing filtering principles help or hurt, and why.

Contributions.

- C1 *Data*. We release 24 synthetic indoor sequences in which surface texture (four levels, quantified by a reproducible texture metric) and scene dynamics (three levels) are varied factorially along identical camera trajectories, with stereo images, RGB-D, ground-truth poses and per-pixel dynamic masks (Sec. III-C).
- C2 *Interaction*. Using these data, we quantify how the effect

of flow-based and geometry-based dynamic-point filtering on ORB-SLAM2 changes with texture, reporting ATE, RPE, tracking completeness and surviving static features over $N_r = 5$ runs per configuration (Sec. IV-A).

C3 *Mechanism*. Using the ground-truth masks, we measure each filter’s per-keypoint dynamic-point precision and recall and its static-feature false-removal rate, and test whether these explain the accuracy differences between filters (Sec. IV-B); in our data they do, but not in the way we hypothesised. As an additional analysis, we compare stereo and RGB-D input, whose depth sources depend differently on texture (Sec. IV-C).

Relation to prior work by the author. This paper grows out of the author’s MSc dissertation [11], which compared the same two filtering principles on a single indoor sequence without controlling texture. The study design, the data, the implementations, all experiments and the text are new; no result of the dissertation is reused.

II. RELATED WORK

Dynamic visual SLAM. DynaSLAM [4] extends ORB-SLAM2 (monocular, stereo and RGB-D) with Mask R-CNN segmentation and, for RGB-D, a multi-view geometry test that projects points from overlapping keyframes and flags them by depth discrepancy and parallax angle; it also inpaints the occluded background. DS-SLAM [3] combines ORB-SLAM2 with SegNet and a moving-consistency check that tracks features with Lucas–Kanade optical flow [12] and tests them against the epipolar constraint. Detect-SLAM [13] runs an SSD detector on keyframes and propagates moving probabilities to map points. FlowFusion [14] segments dynamic regions from PWC-Net optical flow in dense RGB-D SLAM, and ReFusion [10] detects dynamics from TSDF residuals and released the Bonn RGB-D Dynamic dataset. DynaPix SLAM [15] estimates per-pixel motion probabilities without semantics and feeds them into ORB-SLAM2’s map-point selection and bundle adjustment, evaluating on the synthetic GRADE data and on TUM RGB-D. Our FLOW and GEOM filters (Sec. III-B) are generic, semantics-free instances of the DS-SLAM and DynaSLAM geometric principles; we study them as isolated factors rather than as complete systems.

Low-texture SLAM. Feature scarcity is addressed by point-and-line systems [16], [17], by structural and planar constraints [18], [19], by dedicated weak-texture pipelines such as RWT-SLAM [20], by direct and semi-direct methods [21], [22], and by learned dense methods such as DROID-SLAM [23]. SID-SLAM [22] also released the DLR Minimal Texture dataset of real RGB-D sequences with motion-capture ground truth. In these works texture is typically a *categorical* condition (e.g., the TUM structure/texture sequences [9], OpenLORIS corridors [24] or dedicated minimal-texture recordings) rather than a continuous, controlled variable. Evaluation studies [5], [25] compare systems across challenge categories; Bujanca et al. [5] report texture and dynamics as separate categories, not crossed.

TABLE I
POSITIONING. “NO” MEANS THAT NO EXPLICIT, GRADED CONTROL IS REPORTED IN THE SOURCES WE CONSULTED; N/S: NOT STATED IN THOSE SOURCES.

Dataset / study	Synth.	Texture ctrl.	Dynamics ctrl.	Dyn. masks
TUM RGB-D [9]	no	categ. ^a	categ. ^b	no
Bujanca et al. [5]	mixed	separate ^c	separate ^c	n/s
TartanAir [27]	yes	no	not graded	seg. only ^d
VIODE [28]	yes	no	4 levels	yes
GRADE [29]	yes	no	n/s	n/s
CSE [31]	yes	no	static/dyn. twins	n/s
Robust-SLAM [32]	yes	no	static only	–
IL-/SR-/SFSE-SLAM ^e	no	no	no	n/s
This work	yes	4 levels + metric	3 levels	yes

^aStatic structure/texture sequences. ^bSeparate sitting/walking recordings. ^cReported as separate challenge categories, not crossed. ^dSegmentation is provided; dynamic-point labels are not stated. ^eEvaluations reported in [6]–[8] (TUM RGB-D, Bonn, real robot runs).

Feature-scarcity-aware dynamic SLAM. IL-SLAM [6] triggers line features when a feature-awareness criterion indicates that too few points remain after dynamic removal; it is evaluated on TUM sequences. SR-SLAM [7] computes a frame reliability score from detection confidence, spatial distribution, feature richness on a 3×3 grid and depth quality, and falls back to direct photometric and depth refinement when features are scarce; it is evaluated on TUM, Bonn and real robot runs. SFSE-SLAM [8] adds texture-aware compensation through a threshold driven by global feature density, evaluated on TUM and Bonn. These papers motivate our question; our study is complementary, since it does not add a mechanism but measures the underlying effect under controlled texture.

Synthetic data and benchmarks. ICL-NUIM [26] provides static synthetic indoor scenes with largely low-texture walls. TartanAir [27] provides photo-realistic sequences with stereo, depth, segmentation, optical flow and moving objects. VIODE [28] renders each of three environments at four dynamic levels that progressively add moving objects, with instance masks. GRADE [29] generates indoor dynamic scenes with animated humans in Isaac Sim, and a companion study [30] found that DynaSLAM achieved good ATE on such data while losing track for at least about 27 s, i.e. ATE alone can hide failures. The CSE dataset [31] provides static and dynamic versions of service environments with identical trajectories. Robust-SLAM [32] perturbs Replica scenes with image, depth and trajectory corruptions under an explicitly static-world assumption, and VSLAM-LAB [33] offers a unified pipeline for re-running baselines. Table I summarises which of these control texture and dynamics; in the sources we consulted we found none that varies both factorially along the same trajectory.

III. STUDY DESIGN

A. Backbone

We use the public ORB-SLAM2 implementation [2] in its stereo and RGB-D modes with the default tracking, local-mapping and loop-closing threads and with $N_f = 1000$ ORB features [34] per frame (scale factor 1.2, 8 levels, FAST

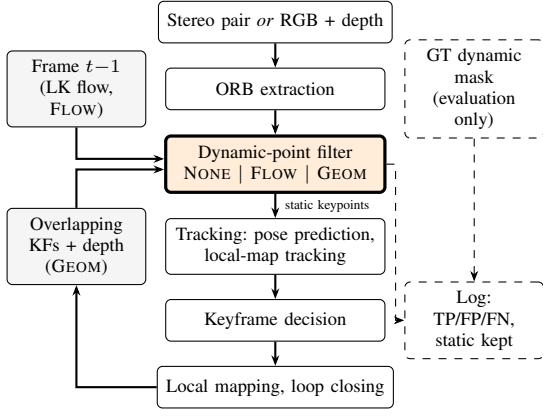


Fig. 1. Where the dynamic-point filter sits in the ORB-SLAM2 tracking front-end (schematic drawn for this paper). Flagged keypoints are removed before tracking and map-point creation. Dashed elements are used only for evaluation (Sec. III-D) and never influence the tracker.

thresholds 20/7, close/far depth threshold 40 baselines); all other parameters are left at their defaults. We use commit f2e6f51 of the public repository, built headless (no viewer) on Windows with MSVC 19.44 and OpenCV 4.10; the port changes only portability code (OpenCV 4 headers, a variable-length array, an allocator type, sleeping) and adds a binary vocabulary loader, and the released patch lists every change. A dynamic-point filter is inserted between ORB extraction and tracking (Fig. 1): keypoints flagged as dynamic are excluded from pose prediction, local-map tracking (motion-only bundle adjustment) and map-point creation. The filter only removes keypoints; it does not add features, change the ORB extractor, or alter the back-end. For evaluation only, each keypoint decision is logged together with its ground-truth label.

B. Dynamic-point filters

Both filters produce a binary label $\hat{y}_k \in \{0, 1\}$ (1 = dynamic) for every ORB keypoint $\mathbf{x}_k$ of the current frame t . Homogeneous coordinates are denoted by $\tilde{\cdot}$.

FLOW: optical flow with an epipolar residual test (after the moving-consistency check of DS-SLAM [3], without semantic segmentation). Each keypoint $\mathbf{x}_k$ of frame t is tracked backwards into frame $t-1$ with pyramidal Lucas–Kanade flow [12] ($w_{LK} = 21$ px window, $\ell_{LK} = 3$ levels), giving $\mathbf{x}'_k$. Pairs with a large LK error ($> e_{LK} = 20$, mean absolute patch difference) or within $b = 10$ px of the image border are discarded and left unlabelled ($\hat{y}_k = 0$). A fundamental matrix $\mathbf{F}_t$ is estimated from the remaining pairs with RANSAC [35] (OpenCV `findFundamentalMat`, inlier threshold $\tau_F = 1$ px, confidence 0.99); with fewer than 15 pairs the frame is not filtered. With the epipolar line $\mathbf{l}_k = \mathbf{F}_t \tilde{\mathbf{x}}'_k = (l_1, l_2, l_3)^\top$, the residual is

$$d_k = \frac{|\tilde{\mathbf{x}}_k^\top \mathbf{F}_t \tilde{\mathbf{x}}'_k|}{\sqrt{l_1^2 + l_2^2}}, \quad \hat{y}_k = [d_k > \tau_{\text{epi}}], \quad (1)$$

with $\tau_{\text{epi}} = 1$ px. Optionally, flagged keypoints are dilated into a region mask of radius r_f and all keypoints inside it are removed; $r_f = 0$ gives a purely point-wise filter, $r_f > 0$ a

region-level filter as used by mask-based methods. We report the point-wise variant, $r_f = 0$. Note that $\mathbf{F}_t$ is estimated from the same keypoints it is used to test, so its quality depends on the number and spatial spread of static correspondences.

GEOM: multi-view depth consistency (after the geometric branch of DynaSLAM [4], without Mask R-CNN). Given a preliminary pose $\hat{\mathbf{T}}_t$ from ORB-SLAM2’s standard tracking with its robust kernel, we select the $K_o = 5$ keyframes with the largest overlap (shared map points) with frame t . Every keyframe keypoint with valid depth is back-projected to a 3D point $\mathbf{X}$ and projected into frame t , giving pixel $\mathbf{x}'$ and projected depth z_{proj} . Let α be the angle between the viewing rays of $\mathbf{X}$ from the keyframe and from frame t , and z' the depth measured in frame t at $\mathbf{x}'$; to avoid seeds caused by sub-pixel projection errors at depth edges, z' is the *largest* valid depth in a 5×5 patch around $\mathbf{x}'$ (RGB-D) or among current keypoints with stereo depth within 3 px of $\mathbf{x}'$ (stereo). Points with $\alpha > \alpha_{\text{max}} = 30^\circ$ are skipped because of possible self-occlusion; otherwise

$$\Delta z = z_{\text{proj}} - z', \quad \mathbf{x}' \text{ is a dynamic seed if } \Delta z > \tau_z, \quad (2)$$

with $\tau_z = 0.30$ m, i.e., something closer than the expected static surface now occupies the pixel. In RGB-D mode the seeds are grown into a mask M_t^{geo} on the depth image (4-neighbours with depth difference $< \tau_g = 0.02$ m, at most 30 px from the seed) and keypoints inside the mask are flagged. In stereo mode only keypoint depths are available, so keypoints within $r_g = 15$ px of a seed are flagged. Flagged keypoints are removed after the preliminary pose has been computed and before local-map tracking, so they enter neither the final pose optimisation nor new map points. GEOM thus depends on depth availability, which in stereo mode is itself texture-dependent, and FLOW depends on the conditioning of $\mathbf{F}_t$.

Implementation and parameter choice. Both filters were implemented for this study as a single additional source file in ORB-SLAM2’s tracking thread (released with the patch); they are not the implementations used in the MSc dissertation. All parameters were fixed before the grid was run, using one texture-rich sequence (scene 1, L0/D1) only to check that the filters behaved as intended. On that sequence a first version (nearest-pixel z' , 60 px growth limit) discarded 18 % of static keypoints in RGB-D mode (12 % with the edge-robust z' alone); the 30 px limit reduced this to 5 %. No parameter was changed after the grid was started.

C. Controlled data generation

Scenes and rendering. We render $S = 2$ furnished indoor scenes, an office ($7 \times 6 \times 2.8$ m) and a laboratory/kitchen ($8 \times 5 \times 3$ m), built from box and cylinder meshes with procedurally generated, tileable grey-scale textures (plaster, wood, tiles, brick, fabric, carpet, marble, posters, book spines). Images are produced by a purpose-written CPU ray caster (Open3D 0.20 `RaycastingScene`, Embree) with Lambertian shading from three point lights, ambient light, no shadows or global illumination, 2×2 supersampling, display gamma, and additive Gaussian noise ($\sigma = 1$ grey level) whose

realisation is identical in every condition. This renderer is far simpler than photo-realistic simulators; we chose it because it makes every factor exactly controllable and runs on a CPU. For each scene one camera trajectory (hand-held style, 15 s at 30 Hz, 450 frames, about 6.0 m path length) is scripted and replayed identically for every texture and dynamics condition, so that viewpoint and scene geometry are held fixed.

Texture levels. Starting from the original materials (level L0), the albedo of every static surface is blended towards its per-material mean colour $\bar{A}$,

$$A_\ell(\mathbf{u}) = (1 - \beta_\ell) A(\mathbf{u}) + \beta_\ell \bar{A}, \quad \beta_\ell \in \{0, \beta_1, \beta_2, 1\}, \quad (3)$$

for levels L0–L3 ($\beta_1 = 0.4$, $\beta_2 = 0.7$, chosen on 15 sub-sampled frames per scene so that $\bar{N}_F$ falls to roughly one half and one fifth of L0); geometry, lighting and the dynamic agents’ appearance are unchanged. Level L3 therefore retains only shading, geometric edges and object boundaries. Because β_ℓ is a rendering parameter rather than a perceptual quantity, each level is characterised by two image-based metrics computed on static pixels only (outside the ground-truth dynamic mask M_t): (i) the mean FAST corner count

$$\bar{N}_F = \frac{1}{T} \sum_{t=1}^T |\{\mathbf{u} \notin M_t : \text{FAST9}(I_t, \mathbf{u}; t_F)\}|, \quad (4)$$

with a fixed threshold $t_F = 20$ on the full-resolution image and no adaptive lowering (unlike ORB-SLAM2’s extractor, whose adaptive threshold would mask texture differences); and (ii) the mean gradient-magnitude entropy

$$\bar{H}_\nabla = \frac{1}{T} \sum_{t=1}^T \left(- \sum_{b=1}^B p_{t,b} \log_2 p_{t,b} \right), \quad (5)$$

where $p_{t,b}$ is the fraction of static pixels whose Sobel gradient magnitude $\|\nabla I_t(\mathbf{u})\|$ falls into bin b of $B = 64$ equal bins over $[0, g_{\max}]$, $g_{\max} = 512$ (larger values fall into the last bin). Both are reported per level in Table II.

Dynamic levels. D0 has no moving agent, D1 one and D2 three. Agents are rigid, human-sized mannequins (1.75 m, assembled from boxes, with strongly textured clothing) that walk at 0.9–1.2 m/s along closed scripted paths; they are not articulated, and their textures are not changed with the texture level. The D1 agent is also present in D2, and every agent follows the same scripted path at every texture level. Because agent count does not fix image coverage, we report the mean dynamic-pixel ratio $\bar{\rho} = \frac{1}{T} \sum_t |M_t|/|\Omega|$ per sequence (Table II). The D0 cells serve as a control: any keypoint removed there is by construction a false positive.

Sensors and ground truth. Each sequence contains a rectified stereo pair (baseline $b_s = 0.10$ m, resolution 640×480 , focal length 400 px), a registered depth image of the left camera rendered from the z-buffer without a depth-noise model, ground-truth camera poses, and per-pixel dynamic masks. In total the design has $S \times 4 \times 3 = 24$ sequences.

TABLE II
TEXTURE AND DYNAMICS STATISTICS PER LEVEL (MEDIAN OVER THE SIX SEQUENCES OF EACH LEVEL). KEYPOINTS ARE ORB-SLAM2 LEFT-IMAGE KEYPOINTS PER FRAME IN NINE RUNS (IDENTICAL FOR STEREO AND RGB-D INPUT); THE LAST COLUMN IS THE FRACTION OF THEM WITH A VALID STEREO DEPTH.

	β_ℓ	$\bar{N}_F$	$\bar{H}_\nabla$ [bits]	keypoints	stereo depth [%]
L0	0.0	326	2.65	1003	50
L1	0.4	151	2.22	993	54
L2	0.7	39	1.73	878	63
L3	1.0	29	1.01	399	71
Mean dynamic-pixel ratio $\bar{\rho}$: D0: 0 D1: 1.9 % D2: 10.5 %					

D. Metrics

Trajectory accuracy. Let $\mathcal{T}$ be the set of frames for which the tracker output a pose, $\mathbf{P}_i$ the estimated and $\mathbf{Q}_i$ the ground-truth pose. Since stereo and RGB-D are metric, we align with the SE(3) transform $\mathbf{S}^*$ that minimises the translational residual [36] (no scale), and compute

$$\text{ATE} = \left(\frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \|\text{trans}(\mathbf{Q}_i^{-1} \mathbf{S}^* \mathbf{P}_i)\|^2 \right)^{1/2}. \quad (6)$$

The translational RPE is computed over all pose pairs 1 m apart along the trajectory. Both are computed with evo [37] (v1.37).

Robustness. ATE over tracked frames can hide failures [30], so we also report the tracking completeness $C = |\mathcal{T}|/T$, the number of tracking losses and re-initialisations N_{lost} , and the mean number of static keypoints used as tracking inliers per frame, $\bar{n}_s$. Runs with $C < C_{\min} = 0.8$, and runs in which ORB-SLAM2 never initialised, are counted as failures; their ATE is excluded from the medians, and failures are reported next to every ATE value.

Filter decisions. A keypoint is ground-truth dynamic ($y_k = 1$) if it lies inside M_t (dilated by $\delta = 2$ px to absorb boundary effects). Over all frames of a run, with TP, FP, FN and TN counted on the keypoints of each frame before removal (frames in which a filter could not run, e.g. GEOM before a map exists, count as all-static decisions),

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FRR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad (7)$$

where the static-feature false-removal rate FRR is the fraction of truly static keypoints that the filter discards. Such per-keypoint labels are not available in real benchmarks such as TUM RGB-D.

E. Protocol and statistics

Because ORB-SLAM2 is not deterministic across runs, every configuration (scene $\times$ texture $\times$ dynamics $\times$ filter $\times$ sensor) is run $N_r = 5$ times, giving 720 runs. We report medians over runs and scenes. The effect of filter f in a cell is $\Delta \text{ATE}_f = \text{med ATE}(f) - \text{med ATE}(\text{NONE})$; negative values mean the filter helps. Within a cell, run distributions are compared with the two-sided Wilcoxon rank-sum test [38]; across cells, paired comparisons of cell medians use the

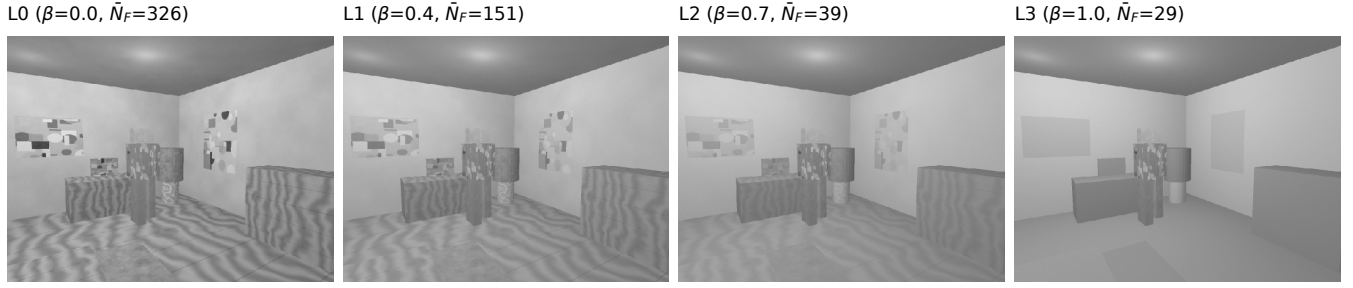


Fig. 2. Left-camera frames of scene 1 at texture levels L0–L3 (dynamic level D2, frame 180), rendered from the same pose. Titles give β_ℓ and the median $\bar{N}_F$ over all sequences of that level.

Wilcoxon signed-rank test, and p -values are corrected with the Holm procedure [39]. The trend of ΔATE_f with texture is summarised by Spearman’s rank correlation with $\bar{N}_F$ across per-scene cells (median over the five runs of one scene). All runs were executed on one Windows laptop (8 logical cores, 16 GB RAM), three at a time; frames were fed in order without dropping, but tracking took 92 ms per frame (median), i.e. slower than the 33 ms frame period. Scripts, logs and raw trajectories are released (Sec. VI).

IV. EXPERIMENTS

Hypotheses. The design is built to test three pre-stated hypotheses; each may be refuted, and refutations will be reported as such.

- H1** The accuracy benefit of dynamic-point filtering ($-\Delta\text{ATE}_f$) decreases as texture decreases, and may become negative at the lowest texture level, because filtering removes a larger share of the scarce static keypoints.
- H2** GEOM has a lower static-feature false-removal rate than FLOW, and the gap widens as texture decreases, because F_t in Eq. (1) is estimated from fewer and less well-distributed static correspondences; this difference in FRR, rather than a difference in recall, explains the accuracy gap between the filters.
- H3** The interaction differs between sensors: in stereo mode, depth availability itself degrades with texture, so the advantage of GEOM over FLOW at low texture is smaller than in RGB-D mode, whose depth is texture-independent.

A. Main grid: texture $\times$ dynamics

Table III lists median ATE, tracking completeness and failed runs for all cells, and Fig. 3 shows ΔATE_f as heat maps, one per filter and sensor, with texture on the vertical and dynamics on the horizontal axis. Completeness and failures are reported next to ATE so that an ATE improvement obtained by tracking fewer frames is not mistaken for a real gain. Under **H1**, the heat maps would show the largest benefit at L0/D2 and a benefit shrinking towards L3; the D0 column isolates the pure cost of filtering, since there is nothing to remove.

Three observations frame the result. First, dynamics dominate: at L0 one agent costs NONE only a few millimetres, but three agents (10.5 % of the pixels) raise its median ATE to 55 cm in both modes, because the tracker partly follows the agents. Second, the pure cost of filtering is small: in the D0 column $|\Delta\text{ATE}_f| \leq 1.5$ cm and completeness is unchanged wherever ORB-SLAM2 initialises. Third, at L3 the static scene alone cannot carry ORB-SLAM2: L3/D0 frames yield only 233–288 ORB keypoints (median per scene), below the more than 500 that ORB-SLAM2’s stereo/RGB-D initialisation requires, so none of the 60 L3/D0 runs initialised, and at L3/D1–D2 initialisation succeeded only because of the textured agents. Only one cell is significant after Holm correction (GEOM, RGB-D, L0/D2: -52 cm, $p_{\text{Holm}} = 0.017$); with ten runs per cell and bimodal outcomes (the tracker either ignores or follows an agent), the remaining differences are descriptive.

H1 is not supported for accuracy and only partly supported for robustness. Across the D1–D2 cells, the rank correlation between the benefit $-\Delta\text{ATE}_f$ and $\bar{N}_F$ was $\rho = 0.49$ (FLOW, RGB-D, $p = 0.09$), 0.34 (FLOW, stereo, $p = 0.23$), -0.52 (GEOM, RGB-D, $p = 0.10$) and 0.44 (GEOM, stereo, $p = 0.18$); none is significant, and GEOM’s RGB-D benefit was in fact largest at L1–L2 (-58 cm at L2/D1). At L3, however, filtering lowered tracking completeness: relative to NONE, the median C in D1–D2 cells dropped by 0.39 (RGB-D) and 0.36 (stereo) with GEOM, which produced a single successful L3 run, whereas FLOW left C essentially unchanged (-0.01) but did not improve ATE ($+9$ cm RGB-D, $+23$ cm stereo at L3/D1).

B. Mechanism: what the filters remove

Table IV reports, for the dynamic level D2 and for each texture level, the per-keypoint precision P, recall R and false-removal rate FRR of each filter, together with the surviving static inliers $\bar{n}_s$ (for NONE this is the reference). The D0 cells add a pure-cost view: there, FRR is the fraction of all keypoints a filter removes for no reason. Fig. 4(a) plots FRR against texture level. Under **H2**, FLOW’s FRR would rise more steeply than GEOM’s as texture decreases, and ΔATE_f would be better predicted by FRR (or by $\bar{n}_s$) than by recall.

TABLE III

MAIN GRID. EACH CELL: MEDIAN ATE [CM] OVER THE SUCCESSFUL RUNS OF 2 SCENES $\times$ 5 RUNS / MEDIAN TRACKING COMPLETENESS C [%] OVER ALL 10 RUNS / NUMBER OF FAILED RUNS ($C < C_{\min} = 0.8$ OR NO INITIALISATION), FOR EVERY TEXTURE LEVEL (ROWS) AND DYNAMIC LEVEL $\times$ FILTER (COLUMNS). -: FEWER THAN THREE SUCCESSFUL RUNS. RPE AND PER-SCENE VALUES ARE GIVEN IN THE RELEASED RESULT FILES.

Sensor	Texture	D0 (static)			D1			D2		
		NONE	FLOW	GEOM	NONE	FLOW	GEOM	NONE	FLOW	GEOM
RGB-D	L0	1.2/100/0	1.5/100/0	1.6/100/0	1.5/100/0	1.4/100/0	1.6/100/0	54.8/100/0	29.4/100/0	2.7/100/0
	L1	1.7/100/0	1.5/100/0	1.8/100/0	21.8/100/0	6.1/100/0	2.2/100/0	80.7/100/0	59.0/100/0	21.4/100/0
	L2	2.6/100/0	2.9/100/0	4.1/100/0	69.7/100/0	78.6/100/0	11.5/100/0	87.9/100/2	90.5/100/3	47.4/68/6
	L3	-/0/10	-/0/10	-/0/10	57.5/73/5	66.4/73/5	-/19/9	-/62/9	-/57/10	-/9/10
Stereo	L0	0.8/100/0	0.8/100/0	0.8/100/0	0.9/100/0	0.9/100/0	0.9/100/0	55.0/100/0	36.3/100/0	53.2/100/0
	L1	0.8/100/0	0.8/100/0	0.9/100/0	1.7/100/0	1.2/100/0	6.6/100/0	48.3/100/0	68.5/100/0	82.3/100/0
	L2	1.1/100/0	1.4/100/0	1.2/100/0	68.1/100/0	68.5/100/0	67.4/100/0	164/100/0	105/100/1	121/96/3
	L3	-/0/10	-/0/10	-/0/10	94.7/100/0	118/99/0	-/49/10	245/57/6	-/57/8	-/51/10

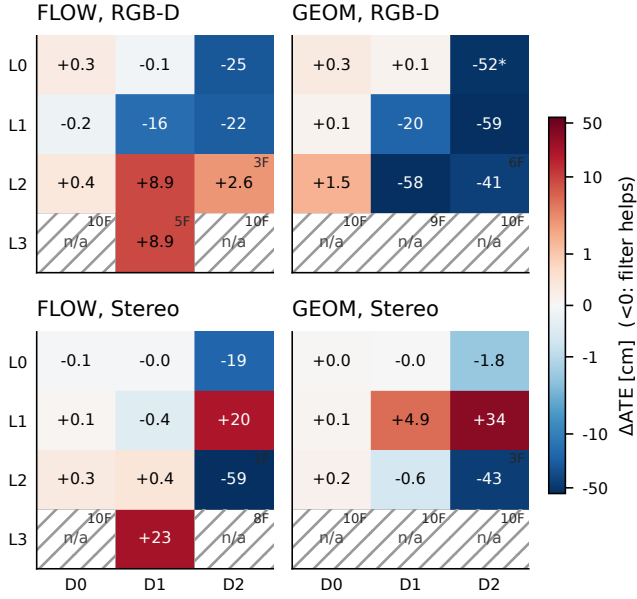


Fig. 3. Effect of filtering, $\Delta\text{ATE}_f = \text{med ATE}(f) - \text{med ATE}(\text{NONE})$ in cm (medians over 2 scenes $\times$ 5 runs, failed runs excluded; negative = filter helps; symmetric-log colour scale centred at 0), per texture level (rows, L0 richest) and dynamic level (columns). *: Holm-corrected Wilcoxon rank-sum $p < 0.05$. kF: failed runs of the filtered configuration (out of 10). n/a: fewer than three successful runs of the filter or of NONE.

H2 is not supported. GEOM discarded more static keypoints than FLOW, not fewer: its FRR was higher in 86 % (RGB-D) and 95 % (stereo) of the scene $\times$ texture $\times$ dynamics cells in which both filters ran (median 6.8 % vs. 1.5 % for RGB-D and 19.6 % vs. 1.5 % for stereo). The predicted trend of FLOW does appear: its FRR at D2 rose from 2.6 % (L0) to 15.8 % (L3) as $\mathbf{F}_t$ was estimated from fewer and less well-spread correspondences, so the FRR gap in favour of FLOW narrowed and at L3 reversed (15.8 % vs. 13.6 % at D2; RGB-D: Spearman $\rho = -0.51$ between the gap and $\bar{N}_F$, $p = 0.014$). The accuracy differences, however, followed recall rather than FRR in RGB-D mode: across D1–D2 cells ΔATE_f correlated with R ($\rho = -0.49$, $p = 0.016$) but not with FRR ($\rho = -0.11$, $p = 0.60$); GEOM found 56–59 % of the dynamic keypoints at

TABLE IV

FILTER DECISIONS AT DYNAMIC LEVEL D2: PRECISION P, RECALL R AND STATIC FALSE-REMOVAL RATE FRR (%), AND STATIC TRACKING INLIERS PER FRAME $\bar{n}_s$; RGB-D MODE, MEDIAN OVER 2 SCENES $\times$ 5 RUNS (RUNS IN WHICH THE FILTER NEVER RAN ARE EXCLUDED). IN STEREO MODE GEOM HAS FRR 15–27 % AND P 21–38 % AT L0–L2 (RELEASED RESULT FILES).

	FLOW				GEOM				NONE
	P	R	FRR	$\bar{n}_s$	P	R	FRR	$\bar{n}_s$	
L0	66	20	2.6	271	71	56	6.2	275	270
L1	61	15	3.5	234	77	59	6.4	252	224
L2	56	10	5.8	133	75	28	6.3	153	132
L3	49	7	15.8	61	71	20	13.6	96	59

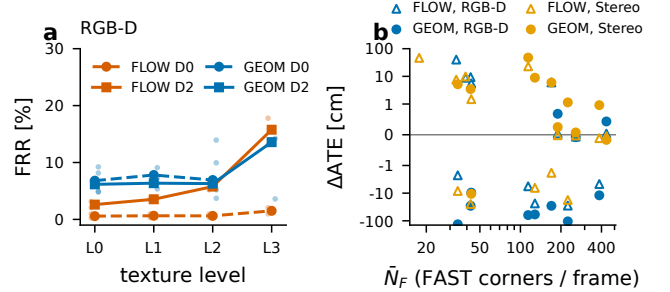


Fig. 4. (a) Static false-removal rate of FLOW and GEOM per texture level (D0 dashed, D2 solid; RGB-D; lines: median over 10 runs, dots: per-scene medians). GEOM at L3/D0 is missing because ORB-SLAM2 never initialised there. (b) ΔATE_f against $\bar{N}_F$ for stereo and RGB-D, one point per scene $\times$ texture $\times$ dynamics cell (D1–D2), symmetric-log axis; cells with fewer than three successful runs are omitted.

L0–L1, FLOW 15–20 %. In stereo mode the pattern reversed: ΔATE_f correlated with FRR ($\rho = 0.40$, $p = 0.046$) and not with R ($\rho = -0.12$), consistent with GEOM’s high stereo FRR (caption of Table IV).

C. Stereo versus RGB-D

Stereo depth in ORB-SLAM2 comes from matching keypoints along epipolar lines and therefore requires texture, whereas rendered RGB-D depth does not. Fig. 4(b) plots ΔATE_f against $\bar{N}_F$ separately for both sensors, and Table II reports the fraction of keypoints with valid stereo depth per

texture level. Under **H3**, the advantage of GEOM over FLOW would decline with texture more strongly in stereo mode than in RGB-D mode.

H3 is supported in outcome but not in mechanism. In RGB-D mode the advantage of GEOM over FLOW ($\Delta\text{ATE}_{\text{FLOW}} - \Delta\text{ATE}_{\text{GEOM}}$, D1–D2 cells) grew as texture decreased (median 3.7, 35 and 38 cm at L0, L1, L2; $\rho = -0.65$ with $\tilde{N}_F$, $p = 0.03$), whereas in stereo mode it was absent or negative at every level (-0.6 , -12 , -14 cm); at L3 neither filter had enough successful runs for a comparison. The hypothesised cause, however, is not what we observe: the fraction of keypoints with valid stereo depth did not fall with texture but rose from 50 % (L0) to 71 % (L3), because the keypoints that remain lie on high-contrast edges and on the agents. The stereo deficit of GEOM is instead associated with its high static false-removal rate at all texture levels, which we conjecture stems from erroneous keyframe stereo depths entering the depth test (not tested here).

V. DISCUSSION AND LIMITATIONS

Relation to feature-scarcity-aware systems. IL-SLAM, SR-SLAM and SFSE-SLAM [6]–[8] motivate their compensation mechanisms by the claim that dynamic-point removal leaves too few static features in low-texture regions. Our controlled data support a weaker form of this claim. At the lowest texture level filtering did reduce completeness, most strongly for GEOM, with only about 60–100 static tracking inliers per frame left at L3/D2 (Table IV). But the accuracy benefit of filtering did not shrink monotonically with texture, and at L2 it was often larger than at L0, because texture loss also made the unfiltered tracker more vulnerable to the agents (NONE at L2/D1: 70 cm vs. 1.5 cm at L0, RGB-D). Moreover, the dominant low-texture failure in our grid occurs before any filter acts: ORB-SLAM2 did not initialise in static L3 scenes at all. A compensation mechanism that only adds features after removal would therefore not address the L3/D0 failures. Our results also contradict the common intuition, which we shared when stating **H2**, that a geometric test is the conservative choice: GEOM removed four to thirteen times more static keypoints than FLOW, and it paid off only where its higher recall mattered and depth was dense and exact (RGB-D).

Statistical strength. Only one of 48 cells is significant after Holm correction, and none of the texture trends reaches $p < 0.01$. The runs are bimodal (an agent is either ignored or followed), so cell medians over ten runs are unstable; the hypothesis tests above should be read as descriptive evidence on two scenes, not as general effect sizes.

Synthetic-to-real gap. Rendered images lack or simplify sensor noise, motion blur, rolling shutter, auto-exposure and depth artefacts, all of which interact with texture. Our renderer is deliberately simple (Lambertian, no shadows or global illumination, exact z-buffer depth), so effect sizes measured here may differ on real data, and the direction of effects is the primary quantity of interest.

Single backbone and generic filters. All results concern ORB-SLAM2 with semantics-free versions of two filtering

principles. Systems with semantic masks, line or planar features, direct alignment or learned correspondences [3], [4], [19], [23], [40] may respond differently to texture; our results do not transfer to them without further experiments.

Few scenes. With $S = 2$ scenes and one trajectory each, differences between scenes cannot be separated from differences in, e.g., room size or furniture layout, and the main tables pool both scenes; per-scene cell values are released with the results.

Texture metric. $\tilde{N}_F$ depends on the FAST threshold and image resolution, and $\tilde{H}_\nabla$ ignores the spatial distribution of gradients; neither captures repetitive texture, which can cause false matches rather than a lack of matches. Albedo blending also leaves shading edges intact, so L3 is *low*-texture rather than texture-free.

Dynamics. The dynamic levels are defined by agent count; image coverage $\bar{\rho}$ and agent speed also matter and are reported but not varied independently. The agents are rigid and keep their texture at every level, so at low texture they supply a growing share of all keypoints; this is realistic for textured clothing but confounds texture with the relative salience of the agents.

Protocol. Filter parameters were fixed on one texture-rich sequence of the grid (Sec. III-B); a held-out tuning scene would have been cleaner. Runs were executed three at a time and slower than real time; ORB-SLAM2’s threads therefore had more time per frame than in a 30 Hz deployment.

VI. CONCLUSION

We have designed a controlled study that separates surface texture from scene dynamics along identical trajectories, in order to measure when dynamic-point filtering helps a feature-based SLAM front-end and why. In two scenes and 720 runs, dynamic-point filtering mattered far more than texture as long as ORB-SLAM2 could initialise, and its benefit did not decrease monotonically with texture (**H1** not supported for accuracy); at the lowest texture level it reduced completeness instead. GEOM was not the more conservative filter (**H2** not supported): it discarded more static keypoints than FLOW, and its RGB-D advantage came from higher recall. Its advantage disappeared in stereo mode, as **H3** predicted, but not because stereo depth became sparse. The data-generation scripts, rendered sequences with ground-truth poses and masks, filter implementations and evaluation scripts are released so that other front-ends can be placed on the same grid.

Data and code availability. The renderer and scene definitions (all textures are procedural, no third-party assets), the ORB-SLAM2 patch with both filters, the run and evaluation scripts, per-run logs, trajectories and result tables are released at <https://github.com/felixxxue/texture-dynamics-slam>. Sequences are regenerated deterministically by the renderer. All code, including the ORB-SLAM2 patch (ORB-SLAM2 is GPLv3), is released under GPL-3.0; the results, logs and figures under CC BY 4.0.

REFERENCES

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A versatile and accurate monocular SLAM system,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [2] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras,” *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [3] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, “DS-SLAM: A semantic visual SLAM towards dynamic environments,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1168–1174.
- [4] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, “DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [5] M. Bujanca, X. Shi, M. Spear, P. Zhao, B. Lennox, and M. Luján, “Robust SLAM systems: Are we there yet?” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 5320–5327.
- [6] H. Zhang, T. N. Canh, C. Li, R. Yang, Y. Ji, and N. Y. Chong, “IL-SLAM: Intelligent line-assisted SLAM based on feature awareness for dynamic environments,” arXiv:2509.02972, 2025.
- [7] H. Zhang, C. Li, T. N. Canh, L. Wang, and N. Y. Chong, “SR-SLAM: Scene reliability-based RGB-D SLAM in diverse environments,” arXiv:2509.01111, 2025.
- [8] Z. Li, Y. Wang, C. Pan, C. Chen, and S. X. Yang, “SFSE-SLAM: Semantic-geometric filtering and texture-aware compensation for robust visual SLAM in dynamic indoor scenes,” *Algorithms*, vol. 19, no. 8, p. 613, 2026.
- [9] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A benchmark for the evaluation of RGB-D SLAM systems,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.
- [10] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, “ReFusion: 3D reconstruction in dynamic environments for RGB-D cameras exploiting residuals,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 7855–7862.
- [11] Z. Xue, “Multi-sensory SLAM on a synthetic dataset,” MSc dissertation, Department of Computer Science, University of Bath, 2022.
- [12] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” in *Proc. 7th International Joint Conference on Artificial Intelligence (IJCAI)*, vol. 2, 1981, pp. 674–679.
- [13] F. Zhong, S. Wang, Z. Zhang, C. Chen, and Y. Wang, “Detect-SLAM: Making object detection and SLAM mutually beneficial,” in *Proc. IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018, pp. 1001–1010.
- [14] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, “FlowFusion: Dynamic dense RGB-D SLAM based on optical flow,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 7322–7328.
- [15] C. Xu, E. Bonetto, and A. Ahmad, “DynaPix SLAM: A pixel-based dynamic visual SLAM approach,” in *Pattern Recognition (DAGM GCPR 2024)*, ser. Lecture Notes in Computer Science, vol. 15298. Springer, 2025, pp. 168–184.
- [16] A. Pumarola, A. Vakhitov, A. Agudo, A. Sanfeliu, and F. Moreno-Noguer, “PL-SLAM: Real-time monocular visual SLAM with points and lines,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 4503–4508.
- [17] R. Gomez-Ojeda, F.-A. Moreno, D. Zuñiga-Noël, D. Scaramuzza, and J. Gonzalez-Jimenez, “PL-SLAM: A stereo SLAM system through the combination of points and line segments,” *IEEE Transactions on Robotics*, vol. 35, no. 3, pp. 734–746, 2019.
- [18] Y. Li, R. Yunus, N. Brasch, N. Navab, and F. Tombari, “RGB-D SLAM with structural regularities,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11 581–11 587.
- [19] R. Yunus, Y. Li, and F. Tombari, “ManhattanSLAM: Robust planar tracking and mapping leveraging mixture of Manhattan frames,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 6687–6693.
- [20] Q. Peng, Z. Xiang, Y. Fan, T. Zhao, and X. Zhao, “RWT-SLAM: Robust visual SLAM for highly weak-textured environments,” arXiv:2207.03539, 2022.
- [21] J. Engel, V. Koltun, and D. Cremers, “Direct sparse odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018.
- [22] A. Fontan, R. Giubilato, L. O. Maza, J. Civera, and R. Triebel, “SID-SLAM: Semi-direct information-driven RGB-D SLAM,” *IEEE Robotics and Automation Letters*, vol. 8, no. 10, pp. 6387–6394, 2023.
- [23] Z. Teed and J. Deng, “DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 16 558–16 569.
- [24] X. Shi, D. Li, P. Zhao, Q. Tian, Y. Tian, Q. Long, C. Zhu, J. Song, F. Qiao, L. Song, Y. Guo, Z. Wang, Y. Zhang, B. Qin, W. Yang, F. Wang, R. H. M. Chan, and Q. She, “Are we ready for service robots? The OpenLORIS-Scene datasets for lifelong SLAM,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 3139–3145.
- [25] D. Sharafutdinov, M. Griguletskii, P. Kopanov, M. Kurenkov, G. Ferrer, A. Burkov, A. Gonnochenko, and D. Tsetsurukou, “Comparison of modern open-source visual SLAM approaches,” arXiv:2108.01654, 2021.
- [26] A. Handa, T. Whelan, J. McDonald, and A. J. Davison, “A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 1524–1531.
- [27] W. Wang, D. Zhu, X. Wang, Y. Hu, Y. Qiu, C. Wang, Y. Hu, A. Kapoor, and S. Scherer, “TartanAir: A dataset to push the limits of visual SLAM,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 4909–4916.
- [28] K. Minoda, F. Schilling, V. Wüest, D. Floreano, and T. Yairi, “VIODE: A simulated dataset to address the challenges of visual-inertial odometry in dynamic environments,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1343–1350, 2021.
- [29] E. Bonetto, C. Xu, and A. Ahmad, “GRADE: Generating realistic and dynamic environments for robotics research with Isaac Sim,” arXiv:2303.04466, 2023.
- [30] —, “Simulation of dynamic environments for SLAM,” arXiv:2305.04286, 2023.
- [31] H. Park, I. Lee, M. Kim, H. Park, and K. Joo, “A benchmark dataset for collaborative SLAM in service environments,” *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 337–11 344, 2024.
- [32] X. Xu, T. Zhang, S. Wang, X. Li, Y. Chen, Y. Li, B. Raj, M. Johnson-Roberson, and X. Huang, “Customizable perturbation synthesis for robust SLAM benchmarking,” arXiv:2402.08125, 2024.
- [33] A. Fontan, T. Fischer, J. Civera, and M. Milford, “VSLAM-LAB: A comprehensive framework for visual SLAM methods and datasets,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 9013–9020.
- [34] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2564–2571.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [36] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [37] M. Grupp, “evo: Python package for the evaluation of odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017.
- [38] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [39] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [40] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.